Books Memo 追加技術設計書

DDL / OpenAPI / ER図 / 画面遷移図 / 実装タスク分解

項目	内容
前提スタック	Java (Spring Boot) + React + PostgreSQL
アプリ形態	個人向け Web アプリ。将来の複数ユーザー化に対応
ISBN連携	openBD を第一優先、未取得時は Google Books を fallback。どちらも未取得なら手動入力
API仕様方針	OpenAPI 3.1.1 形式で REST API を整理
DB設計方針	書誌原本(book_master)と個人利用情報(user_book)を分離
非機能方針	MVP で動作優先。監査ログ・request_id・論理削除を標準実装

本資料は、前回の詳細設計書を実装へ進めるための追加設計パッケージである。DDL は PostgreSQL 前提、API 契約は OpenAPI 形式、画面導線は Web アプリの MVP 実装を想定して整理した。

ISBN 連携は openBD を第一候補、未取得時は Google Books を補完候補とし、どちらでも見つからない場合は手入力へフォールバックする設計としている。

1. DDL(SQL) 設計

1.1 テーブル構成サマリ

Table	Purpose	PK	Main columns
app.users	ユーザー認証・プロフィール	id	email, display_name, role, password_hash
app.book_master	書誌原本メタデータ	id	isbn13, title, authors_json, publisher
app.user_book	ユーザーごとの所持・進捗・評価	id	user_id, book_master_id, status, rating, memo
app.tags	ユーザー定義タグ	id	user_id, name, color_hex, sort_order
app.user_book_tag	本とタグの中間	id	user_book_id, tag_id
app.isbn_lookup_cache	外部ISBN検索キャッシュ	id	isbn13, source_name, response_json, normalized_payload_json
app.activity_log	監査ログ・追跡用	id	user_id, action_type, target_type, request_id

1.2 設計メモ

・users は認証と表示名を保持する。email は LOWER(email) で一意制約を持たせる。

・book_master は再利用可能な書誌原本。外部 API 取得由来を source_primary で残す。

・user_book はユーザー固有の状態・評価・メモを保持し、deleted_at による論理削除を採用する。

・isbn_lookup_cache は API 生レスポンスと正規化結果の両方を保持し、hit_count でキャッシュ利用頻度を追えるようにする。

・updated_at は trigger で自動更新する。

1.3 SQL 全文（実装用）

```
-- ============================================================
-- Books Memo / PostgreSQL DDL
-- Assumption: PostgreSQL 16+ / schema = app
-- ============================================================
```

```
CREATE SCHEMA IF NOT EXISTS app;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- ------------------------------------------------------------
-- generic trigger for updated_at
-- ------------------------------------------------------------
CREATE OR REPLACE FUNCTION app.set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- ------------------------------------------------------------
-- users
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS app.users (
    id               BIGSERIAL PRIMARY KEY,
    email            VARCHAR(255) NOT NULL,
    password_hash    VARCHAR(255) NOT NULL,
    display_name     VARCHAR(100) NOT NULL,
    role             VARCHAR(30)  NOT NULL DEFAULT 'USER'
                     CHECK (role IN ('USER', 'ADMIN')),
    is_active        BOOLEAN      NOT NULL DEFAULT TRUE,
    last_login_at    TIMESTAMPTZ  NULL,
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at       TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX IF NOT EXISTS uq_users_email_lower
    ON app.users (LOWER(email));

CREATE TRIGGER trg_users_updated_at
BEFORE UPDATE ON app.users
FOR EACH ROW EXECUTE FUNCTION app.set_updated_at();

-- ------------------------------------------------------------
-- book_master
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS app.book_master (
    id                      BIGSERIAL PRIMARY KEY,
    isbn13                  CHAR(13) NULL
                            CHECK (isbn13 ~ '^[0-9]{13}$'),
    isbn10                  CHAR(10) NULL
                            CHECK (isbn10 ~ '^[0-9X]{10}$'),
    title                   VARCHAR(500) NOT NULL,
    subtitle                VARCHAR(500) NULL,
    authors_json            JSONB NULL,
    publisher               VARCHAR(255) NULL,

    published_date          DATE NULL,
    language_code           VARCHAR(10) NULL,
    page_count              INTEGER NULL CHECK (page_count IS NULL OR page_count >= 1),
    description             TEXT NULL,
    thumbnail_url           TEXT NULL,
    source_primary          VARCHAR(30) NOT NULL
                            CHECK (source_primary IN ('OPENBD', 'GOOGLE_BOOKS', 'MANUAL')),
    source_last_fetched_at  TIMESTAMPTZ NULL,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX IF NOT EXISTS uq_book_master_isbn13
    ON app.book_master (isbn13)
    WHERE isbn13 IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_book_master_title_trgm
    ON app.book_master USING GIN (title gin_trgm_ops);

CREATE INDEX IF NOT EXISTS idx_book_master_publisher_trgm
    ON app.book_master USING GIN (publisher gin_trgm_ops);

CREATE TRIGGER trg_book_master_updated_at
BEFORE UPDATE ON app.book_master
FOR EACH ROW EXECUTE FUNCTION app.set_updated_at();

-- ------------------------------------------------------------
```

```
-- user_book
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS app.user_book (
    id               BIGSERIAL PRIMARY KEY,
    user_id          BIGINT NOT NULL REFERENCES app.users(id),
    book_master_id   BIGINT NOT NULL REFERENCES app.book_master(id),
    status           VARCHAR(30) NOT NULL
                     CHECK (status IN ('WISHLIST', 'PURCHASED', 'READING', 'FINISHED', 'ON_HOLD', 'DROPPED', 'TSUNDOKU
    rating           NUMERIC(2,1) NULL
                     CHECK (rating IS NULL OR (rating >= 0.5 AND rating <= 5.0 AND MOD((rating * 10)::INTEGER, 5) = 0)
    favorite_flag    BOOLEAN NOT NULL DEFAULT FALSE,
    purchase_date    DATE NULL,
    start_date       DATE NULL,
    finish_date      DATE NULL,
    memo             TEXT NULL,
    location_note    VARCHAR(255) NULL,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    deleted_at       TIMESTAMPTZ NULL,
    CONSTRAINT chk_user_book_dates
        CHECK (finish_date IS NULL OR start_date IS NULL OR finish_date >= start_date)
);

CREATE UNIQUE INDEX IF NOT EXISTS uq_user_book_active
    ON app.user_book (user_id, book_master_id)
    WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_user_book_user_status_updated

    ON app.user_book (user_id, status, updated_at DESC)
    WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_user_book_user_rating
    ON app.user_book (user_id, rating DESC)
    WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_user_book_user_dates
    ON app.user_book (user_id, start_date, finish_date)
    WHERE deleted_at IS NULL;

CREATE TRIGGER trg_user_book_updated_at
BEFORE UPDATE ON app.user_book
FOR EACH ROW EXECUTE FUNCTION app.set_updated_at();

-- ------------------------------------------------------------
-- tags
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS app.tags (
    id               BIGSERIAL PRIMARY KEY,
    user_id          BIGINT NOT NULL REFERENCES app.users(id),
    name             VARCHAR(50) NOT NULL,
    color_hex        CHAR(7) NULL CHECK (color_hex IS NULL OR color_hex ~ '^#[0-9A-Fa-f]{6}$'),
    sort_order       INTEGER NOT NULL DEFAULT 0,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX IF NOT EXISTS uq_tags_user_name
    ON app.tags (user_id, name);

CREATE INDEX IF NOT EXISTS idx_tags_user_sort
    ON app.tags (user_id, sort_order, id);

CREATE TRIGGER trg_tags_updated_at
BEFORE UPDATE ON app.tags
FOR EACH ROW EXECUTE FUNCTION app.set_updated_at();

-- ------------------------------------------------------------
-- user_book_tag
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS app.user_book_tag (
    id               BIGSERIAL PRIMARY KEY,
    user_book_id     BIGINT NOT NULL REFERENCES app.user_book(id) ON DELETE CASCADE,
    tag_id           BIGINT NOT NULL REFERENCES app.tags(id) ON DELETE CASCADE,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX IF NOT EXISTS uq_user_book_tag
```

```
    ON app.user_book_tag (user_book_id, tag_id);

CREATE INDEX IF NOT EXISTS idx_user_book_tag_tag_id
    ON app.user_book_tag (tag_id, user_book_id);

-- ------------------------------------------------------------

-- isbn_lookup_cache
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS app.isbn_lookup_cache (
    id                        BIGSERIAL PRIMARY KEY,
    isbn13                    CHAR(13) NOT NULL CHECK (isbn13 ~ '^[0-9]{13}$'),
    source_name               VARCHAR(30) NOT NULL
                              CHECK (source_name IN ('OPENBD', 'GOOGLE_BOOKS', 'RAKUTEN')),
    http_status               INTEGER NOT NULL,
    response_json             JSONB NULL,
    normalized_payload_json   JSONB NULL,
    fetched_at                TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at                TIMESTAMPTZ NOT NULL,
    hit_count                 INTEGER NOT NULL DEFAULT 1
                              CHECK (hit_count >= 1)
);

CREATE INDEX IF NOT EXISTS idx_isbn_cache_lookup
    ON app.isbn_lookup_cache (isbn13, source_name, expires_at DESC);

CREATE INDEX IF NOT EXISTS idx_isbn_cache_fetched_at
    ON app.isbn_lookup_cache (fetched_at DESC);

-- ------------------------------------------------------------
-- activity_log
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS app.activity_log (
    id               BIGSERIAL PRIMARY KEY,
    user_id          BIGINT NULL REFERENCES app.users(id),
    action_type      VARCHAR(50) NOT NULL,
    target_type      VARCHAR(50) NOT NULL,
    target_id        BIGINT NULL,
    request_id       VARCHAR(64) NOT NULL,
    detail_json      JSONB NULL,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_activity_log_request_id
    ON app.activity_log (request_id);

CREATE INDEX IF NOT EXISTS idx_activity_log_user_created
    ON app.activity_log (user_id, created_at DESC);

-- ------------------------------------------------------------
-- optional seed example
-- ------------------------------------------------------------
-- INSERT INTO app.users(email, password_hash, display_name)
-- VALUES ('demo@example.com', '$2a$10$replace_me', 'Demo User');
```

2. OpenAPI 定義

2.1 エンドポイント一覧

Method	Path	Purpose	Auth	Main parameters	Responses
POST	/auth/login	ログイン	匿名	email / password	200, 401
POST	/auth/logout	ログアウト	Bearer	-	204
GET	/me	ログインユーザー取得	Bearer	-	200
GET	/books	本一覧検索	Bearer	keyword, status, tagId, favorite, sort, page, size	200
POST	/books	手動登録	Bearer	title 必須	201, 409
POST	/books/import-by-isbn	ISBN 取込登録	Bearer	isbn, status	201, 404
GET	/books/{userBookId}	本詳細	Bearer	path: userBookId	200, 404
PATCH	/books/{userBookId}	本更新	Bearer	status, rating, memo, tagIds ...	200
DELETE	/books/{userBookId}	論理削除	Bearer	path: userBookId	204
PATCH	/books/{userBookId}/status	状態のみ更新	Bearer	status	200
GET	/tags	タグ一覧	Bearer	-	200
POST	/tags	タグ作成	Bearer	name	201
PATCH	/tags/{tagId}	タグ更新	Bearer	name, colorHex, sortOrder	200
DELETE	/tags/{tagId}	タグ削除	Bearer	path: tagId	204
GET	/isbn/{isbn}	ISBN 候補取得のみ	Bearer	path: isbn	200, 404

2.2 API 設計メモ

・認証は bearer token 前提で整理。/auth/login のみ匿名アクセス。

・一覧系は page/size/sort を共通パラメータ化し、MVP でもページネーションを前提とする。

・/books/import-by-isbn は「検索＋登録」をまとめた業務 API。/isbn/{isbn} は候補確認専用で、画面側で候補を見せたい時に使う。

・エラー形式は code / message / requestId を標準化し、障害時の追跡を容易にする。

2.3 OpenAPI YAML（全文）

```
openapi: 3.1.1
info:
  title: Books Memo API
  version: 1.0.0
  summary: Personal reading tracker API
  description: >-
    Books Memo ■■■ REST API■■■■■■(book_master)■■■■■■■■(user_book)■■■■■■
    ISBN ■■■■■■■■■■■■■■■■■■■■■■■■■
servers:
  - url: https://api.example.com/api/v1
    description: production
  - url: http://localhost:8080/api/v1
    description: local
security:
  - bearerAuth: []
tags:
  - name: Auth
  - name: Books
  - name: ISBN
  - name: Tags
```

```yaml
paths:
  /auth/login:
    post:
      tags: [Auth]
      summary: Login
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/LoginRequest'
      responses:
        '200':
          description: Login success
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/LoginResponse'
        '401':
          description: Invalid credentials
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ApiError'
  /auth/logout:
    post:
      tags: [Auth]
      summary: Logout
      responses:
        '204':
          description: Logout success
  /me:
    get:
      tags: [Auth]
      summary: Get current user
      responses:
        '200':
          description: Current user profile
          content:

            application/json:
              schema:
                $ref: '#/components/schemas/MeResponse'
  /books:
    get:
      tags: [Books]
      summary: Search books
      parameters:
        - in: query
          name: keyword
          schema: { type: string, maxLength: 100 }
        - in: query
          name: status
          schema:
            $ref: '#/components/schemas/BookStatus'
        - in: query
          name: tagId
          schema: { type: integer, format: int64 }
        - in: query
          name: favorite
          schema: { type: boolean }
        - in: query
          name: sort
          schema:
            type: string
            enum: [updatedAtDesc, createdAtDesc, titleAsc, ratingDesc, publishedDateDesc]
        - in: query
          name: page
          schema: { type: integer, minimum: 1, default: 1 }
        - in: query
          name: size
          schema: { type: integer, minimum: 1, maximum: 100, default: 20 }
      responses:
        '200':
          description: Paged list
          content:
            application/json:
```

```yaml
            schema:
              $ref: '#/components/schemas/BookListResponse'
  post:
    tags: [Books]
    summary: Create manual book entry
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CreateBookRequest'
    responses:
      '201':
        description: Created
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UserBookDetail'
      '409':
        description: Duplicate active record
        content:
          application/json:
            schema:

              $ref: '#/components/schemas/ApiError'
/books/import-by-isbn:
  post:
    tags: [Books, ISBN]
    summary: Import book by ISBN
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ImportByIsbnRequest'
    responses:
      '201':
        description: Imported
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UserBookDetail'
      '404':
        description: ISBN not found in providers
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ApiError'
/books/{userBookId}:
  get:
    tags: [Books]
    summary: Get detail
    parameters:
      - $ref: '#/components/parameters/UserBookId'
    responses:
      '200':
        description: Detail
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UserBookDetail'
      '404':
        description: Not found
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ApiError'
  patch:
    tags: [Books]
    summary: Update book info
    parameters:
      - $ref: '#/components/parameters/UserBookId'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UpdateBookRequest'
```

```yaml
    responses:
      '200':
        description: Updated
        content:
          application/json:
            schema:

              $ref: '#/components/schemas/UserBookDetail'
  delete:
    tags: [Books]
    summary: Logical delete
    parameters:
      - $ref: '#/components/parameters/UserBookId'
    responses:
      '204':
        description: Deleted
/books/{userBookId}/status:
  patch:
    tags: [Books]
    summary: Update reading status only
    parameters:
      - $ref: '#/components/parameters/UserBookId'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UpdateStatusRequest'
    responses:
      '200':
        description: Updated
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UserBookDetail'
/tags:
  get:
    tags: [Tags]
    summary: List tags
    responses:
      '200':
        description: Tag list
        content:
          application/json:
            schema:
              type: object
              properties:
                items:
                  type: array
                  items:
                    $ref: '#/components/schemas/Tag'
  post:
    tags: [Tags]
    summary: Create tag
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CreateTagRequest'
    responses:
      '201':
        description: Created
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Tag'

/tags/{tagId}:
  patch:
    tags: [Tags]
    summary: Update tag
    parameters:
      - $ref: '#/components/parameters/TagId'
    requestBody:
      required: true
      content:
        application/json:
```

```yaml
            schema:
              $ref: '#/components/schemas/UpdateTagRequest'
      responses:
        '200':
          description: Updated
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Tag'
    delete:
      tags: [Tags]
      summary: Delete tag
      parameters:
        - $ref: '#/components/parameters/TagId'
      responses:
        '204':
          description: Deleted
  /isbn/{isbn}:
    get:
      tags: [ISBN]
      summary: Lookup ISBN candidates without creating user_book
      parameters:
        - in: path
          name: isbn
          required: true
          schema:
            type: string
            pattern: '^[0-9X-]{10,17}$'
      responses:
        '200':
          description: Candidate data
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/IsbnLookupResponse'
        '404':
          description: Not found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ApiError'
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  parameters:
    UserBookId:
      in: path

      name: userBookId
      required: true
      schema: { type: integer, format: int64 }
    TagId:
      in: path
      name: tagId
      required: true
      schema: { type: integer, format: int64 }
  schemas:
    BookStatus:
      type: string
      enum: [WISHLIST, PURCHASED, READING, FINISHED, ON_HOLD, DROPPED, TSUNDOKU]
    LoginRequest:
      type: object
      required: [email, password]
      properties:
        email: { type: string, format: email }
        password: { type: string, minLength: 8, maxLength: 72 }
    LoginResponse:
      type: object
      required: [accessToken, tokenType, expiresIn, user]
      properties:
        accessToken: { type: string }
        tokenType: { type: string, example: Bearer }
        expiresIn: { type: integer, example: 3600 }
        user:
          $ref: '#/components/schemas/MeResponse'
```

```yaml
    MeResponse:
      type: object
      required: [id, email, displayName, role]
      properties:
        id: { type: integer, format: int64 }
        email: { type: string, format: email }
        displayName: { type: string }
        role: { type: string, enum: [USER, ADMIN] }
    BookMasterSummary:
      type: object
      required: [title, sourcePrimary]
      properties:
        id: { type: integer, format: int64 }
        isbn13: { type: string, nullable: true }
        isbn10: { type: string, nullable: true }
        title: { type: string }
        subtitle: { type: string, nullable: true }
        authors: { type: array, items: { type: string } }
        publisher: { type: string, nullable: true }
        publishedDate: { type: string, format: date, nullable: true }
        thumbnailUrl: { type: string, nullable: true }
        sourcePrimary: { type: string, enum: [OPENBD, GOOGLE_BOOKS, MANUAL] }
    Tag:
      type: object
      required: [id, name, sortOrder]
      properties:
        id: { type: integer, format: int64 }
        name: { type: string, maxLength: 50 }
        colorHex: { type: string, pattern: '^#[0-9A-Fa-f]{6}$', nullable: true }
        sortOrder: { type: integer }
    BookListItem:
      type: object
      required: [id, status, favoriteFlag, bookMaster]

      properties:
        id: { type: integer, format: int64 }
        status: { $ref: '#/components/schemas/BookStatus' }
        rating: { type: number, format: double, nullable: true }
        favoriteFlag: { type: boolean }
        purchaseDate: { type: string, format: date, nullable: true }
        startDate: { type: string, format: date, nullable: true }
        finishDate: { type: string, format: date, nullable: true }
        updatedAt: { type: string, format: date-time }
        tags:
          type: array
          items:
            $ref: '#/components/schemas/Tag'
        bookMaster:
          $ref: '#/components/schemas/BookMasterSummary'
    PaginationMeta:
      type: object
      required: [page, size, totalItems, totalPages]
      properties:
        page: { type: integer }
        size: { type: integer }
        totalItems: { type: integer, format: int64 }
        totalPages: { type: integer }
    BookListResponse:
      type: object
      required: [items, meta]
      properties:
        items:
          type: array
          items:
            $ref: '#/components/schemas/BookListItem'
        meta:
          $ref: '#/components/schemas/PaginationMeta'
    UserBookDetail:
      allOf:
        - $ref: '#/components/schemas/BookListItem'
        - type: object
          properties:
            memo: { type: string, nullable: true }
            locationNote: { type: string, nullable: true }
            createdAt: { type: string, format: date-time }
            deletedAt: { type: string, format: date-time, nullable: true }
    CreateBookRequest:
      type: object
```

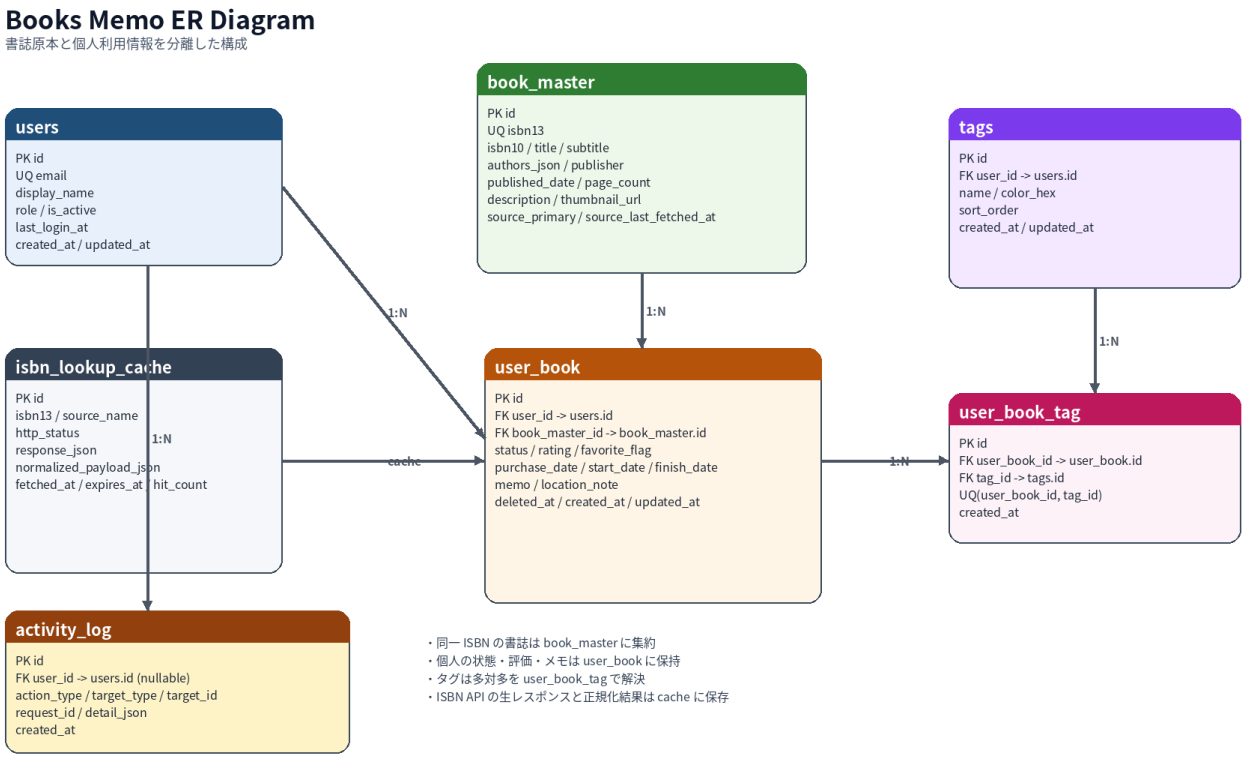
```
    required: [title, status]
    properties:
      isbn13: { type: string, nullable: true }
      isbn10: { type: string, nullable: true }
      title: { type: string, maxLength: 500 }
      subtitle: { type: string, nullable: true }
      authors: { type: array, items: { type: string } }
      publisher: { type: string, nullable: true }
      publishedDate: { type: string, format: date, nullable: true }
      description: { type: string, nullable: true }
      thumbnailUrl: { type: string, nullable: true }
      status: { $ref: '#/components/schemas/BookStatus' }
      rating: { type: number, format: double, nullable: true }
      favoriteFlag: { type: boolean, default: false }
      memo: { type: string, nullable: true }
      tagIds:

        type: array
        items: { type: integer, format: int64 }
  ImportByIsbnRequest:
    type: object
    required: [isbn, status]
    properties:
      isbn: { type: string }
      status: { $ref: '#/components/schemas/BookStatus' }
      memo: { type: string, nullable: true }
      tagIds:
        type: array
        items: { type: integer, format: int64 }
  UpdateBookRequest:
    type: object
    properties:
      status: { $ref: '#/components/schemas/BookStatus' }
      rating: { type: number, format: double, nullable: true }
      favoriteFlag: { type: boolean }
      purchaseDate: { type: string, format: date, nullable: true }
      startDate: { type: string, format: date, nullable: true }
      finishDate: { type: string, format: date, nullable: true }
      memo: { type: string, nullable: true }
      locationNote: { type: string, nullable: true }
      tagIds:
        type: array
        items: { type: integer, format: int64 }
  UpdateStatusRequest:
    type: object
    required: [status]
    properties:
      status: { $ref: '#/components/schemas/BookStatus' }
  CreateTagRequest:
    type: object
    required: [name]
    properties:
      name: { type: string, maxLength: 50 }
      colorHex: { type: string, pattern: '^#[0-9A-Fa-f]{6}$', nullable: true }
      sortOrder: { type: integer, default: 0 }
  UpdateTagRequest:
    type: object
    properties:
      name: { type: string, maxLength: 50 }
      colorHex: { type: string, pattern: '^#[0-9A-Fa-f]{6}$', nullable: true }
      sortOrder: { type: integer }
  IsbnLookupCandidate:
    type: object
    required: [isbn13, title, sourceName]
    properties:
      isbn13: { type: string }
      isbn10: { type: string, nullable: true }
      title: { type: string }
      subtitle: { type: string, nullable: true }
      authors: { type: array, items: { type: string } }
      publisher: { type: string, nullable: true }
      publishedDate: { type: string, format: date, nullable: true }
      thumbnailUrl: { type: string, nullable: true }
      description: { type: string, nullable: true }
      sourceName: { type: string, enum: [OPENBD, GOOGLE_BOOKS, MANUAL] }
      cacheHit: { type: boolean }
  IsbnLookupResponse:
```

```yaml
    type: object
    required: [queryIsbn, candidates]
    properties:
      queryIsbn: { type: string }
      candidates:
        type: array
        items:
          $ref: '#/components/schemas/IsbnLookupCandidate'
  ApiError:
    type: object
    required: [code, message, requestId]
    properties:
      code: { type: string, example: BOOK_DUPLICATE }
      message: { type: string }
      requestId: { type: string }
      details:
        type: array
        items: { type: string }
```

3. ER図

3.1 図解

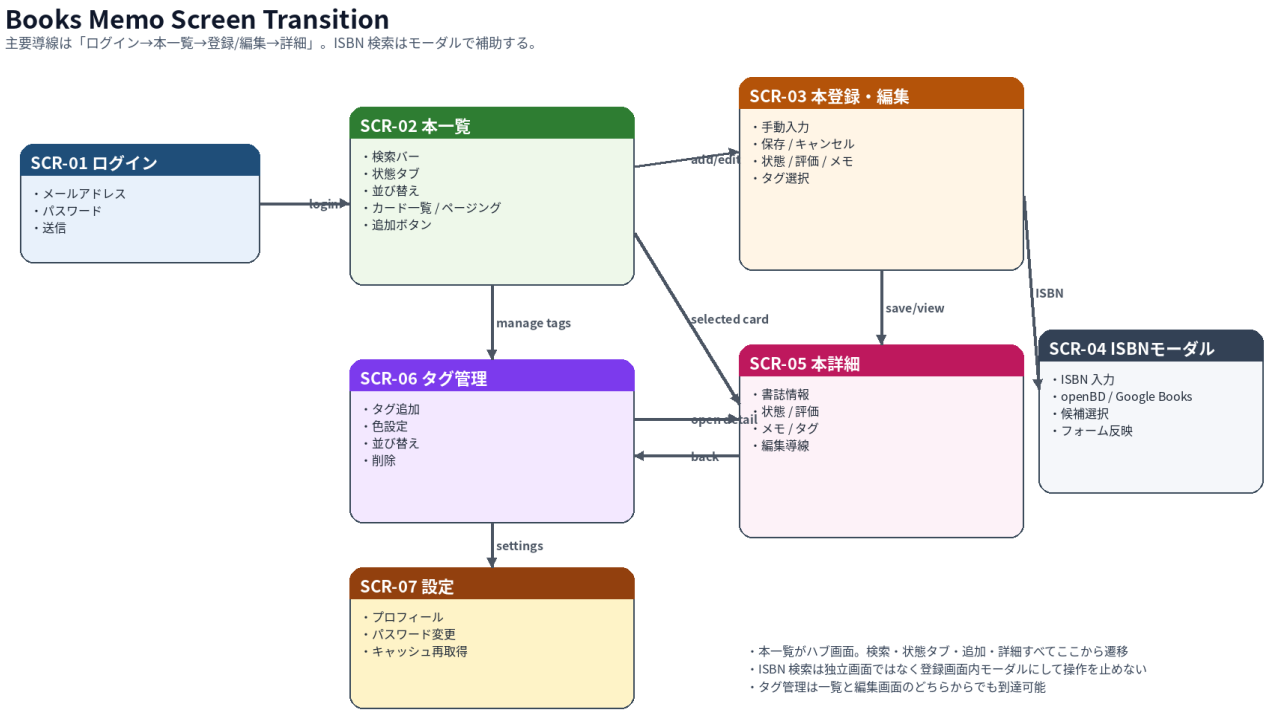


3.2 関係整理

From	Relation	To	Meaning
users	1:N	user_book	1ユーザーが複数の蔵書レコードを持つ
book_master	1:N	user_book	同一書誌原本を複数ユーザーが参照できる
user_book	N:N	tags	user_book_tag を介してタグ付け
isbn_lookup_cache	cache	book_master	API 生レスポンスと正規化データを保持
users	1:N	activity_log	request_id 付きの操作監査

4. 画面遷移図

4.1 図解



4.2 画面ごとの役割

Screen ID	Name	Main role	Transitions
SCR-01	ログイン	初期入口	SCR-02 本一覧
SCR-02	本一覧	検索 / タブ / 並び替え / 一覧 / 追加	SCR-03 編集 / SCR-04 ISBN / SCR-05 詳細 / SCR-06 タグ / SCR-07
SCR-03	本登録・編集	手動入力・保存・キャンセル	SCR-02 本一覧 / SCR-05 詳細
SCR-04	ISBN 検索モーダル	ISBN入力・外部候補取得・フォーム反映	SCR-03 本登録・編集
SCR-05	本詳細	状態 / 評価 / メモ / タグ確認	SCR-03 編集 / SCR-02 本一覧
SCR-06	タグ管理	タグ追加・更新・削除	SCR-02 本一覧 / SCR-03 編集
SCR-07	設定	プロフィール / パスワード / キャッシュ再取得	SCR-02 本一覧

5. 実装タスク分解(WBS)

5.1 フェーズ別の考え方

・Phase P0〜P2 で土台（環境・DB・認証）を固める。

・Phase P3〜P5 が業務の中核（蔵書管理・ISBN 連携・タグ）で、ここまでで API の MVP が成立する。

・Phase P6 以降でフロント統合、E2E、仕上げを行う。

Phase	Category	Task	Days	Deps	Done criteria
P0	環境構築	リポジトリ・CI・Spring Boot/React 初期化	0.5		着手可能
P0	環境構築	Docker Compose(PostgreSQL)・env 整備	0.5	P0-1	ローカル起動
P1	DB/DDL	DDL 実装・マイグレーション作成	1.0	P0	テーブル生成
P1	DB/DDL	updated_at trigger・初期 index 作成	0.5	P1-1	DDL 完了
P2	認証	login / logout / me API 実装	1.0	P0	認証通過
P2	認証	JWT/Session 方針確定・ガード実装	0.5	P2-1	保護ルート可
P3	書誌管理	book_master / user_book entity/repository/service	1.0	P1	基礎 CRUD 可
P3	書誌管理	本一覧 API + 検索 + ページング	1.0	P3-1	一覧取得可
P3	書誌管理	本詳細 API + 更新 + 論理削除	1.0	P3-2	詳細更新可
P4	ISBN連携	ISBN 正規化・openBD クライアント	1.0	P3	openBD 応答確認
P4	ISBN連携	Google Books fallback 実装	0.5	P4-1	fallback 可
P4	ISBN連携	isbn_lookup_cache 実装	0.5	P4-1	cache 再利用可
P4	ISBN連携	import-by-isbn API 実装	0.5	P4-1,P4-2,P4-3	取込登録可
P5	タグ管理	tags / user_book_tag CRUD	0.5	P1	タグ登録可
P5	タグ管理	一覧フィルタ連携	0.5	P5-1,P3-2	タグ絞込可
P6	Frontend	ログイン画面作成	0.5	P2	ログインUI
P6	Frontend	本一覧画面・検索バー・タブ	1.0	P3-2	一覧UI
P6	Frontend	本登録/編集画面	1.0	P3-3	編集UI
P6	Frontend	ISBN モーダル + 候補反映	0.5	P4-4	ISBN UI
P6	Frontend	タグ管理・設定画面	0.5	P5	補助画面
P7	品質	API テスト・バリデーション確認	1.0	P2,P3,P4,P5	主要API緑
P7	品質	E2E シナリオ（ログイン→登録→検索→更新→削除）	1.0	P6	主要導線OK
P8	仕上げ	エラー文言・ローディング・空状態	0.5	P6,P7	UI 調整
P8	仕上げ	README・ER図・OpenAPI 公開	0.5	P7	公開準備

総見積りの目安: 17.0 人日 / 136.0 時間（単独実装・MVP 基準）。

6. 参考URL

Reference	URL
OpenAPI FAQ / latest version	https://www.openapis.org/faq
OpenAPI spec 3.1.1	https://spec.openapis.org/oas/v3.1.1.html
PostgreSQL CREATE TABLE	https://www.postgresql.org/docs/current/sql-createtable.html
PostgreSQL data types	https://www.postgresql.org/docs/current/datatype.html
openBD official top	https://openbd.jp/
Google Books API usage	https://developers.google.com/books/docs/v1/using

Excel 側には、SQL / YAML を行番号つきで再編集しやすい形でも収録している。